

파이썬 함수(Function)

[1차시] 함수 정의와 호출 기초

📖 핵심 개념 설명

자주 사용하는 안내문이나 코드를 매번 새롭게 작성하면 비효율적입니다. 파이썬에서는 자주 쓰는 동작을 def 키워드를 이용해 하나의 블록으로 묶어둘 수 있습니다. 이를 함수 정의라고 하며 정의된 함수는 이름을 호출해야만 비로소 실행됩니다. 데이터 분석 동아리의 시작 알림 코드를 함수로 만들어 봅시다.

📄 전체 실습 코드

```
def show_welcome():  
    print("=====")  
    print("데이터 분석 동아리에 오신 것을 환영합니다!")  
    print("=====")  
  
# 함수를 호출하여 화면에 출력하기  
show_welcome()  
show_welcome()
```

🔍 코드 훑아보기

- **def show_welcome():** : show_welcome이라는 이름의 함수를 새롭게 정의합니다. 뒤에 콜론을 붙여 블록의 시작을 알립니다.
- **들여쓰기 적용** : 콜론 다음 줄부터 들여쓰기된 모든 print 문장들은 하나의 함수 상자 안에 포장됩니다.
- **show_welcome()** : 포장해둔 함수 상자를 실행시키는 호출 명령어입니다. 두 번 적었으므로 안내문이 두 번 출력됩니다.

[2차시] 매개변수(Parameter)를 통한 데이터 전달

📌 핵심 개념 설명

언제나 똑같은 결과만 보여주는 함수보다 외부에서 데이터를 입력받아 상황에 맞게 동작하는 함수가 훨씬 유용합니다. 함수를 정의할 때 괄호 안에 매개변수라는 빈 그릇을 만들어두면 외부에서 인수라는 실제 데이터를 던져줄 수 있습니다. 특정 브랜드의 PPL 광고 분석 시작을 알리는 함수를 만들어 봅시다.

📄 전체 실습 코드

```
def analyze_ppl(brand):  
    print(brand, "브랜드의 PPL 광고 효과 분석을 시작합니다.")  
    print("데이터 수집 및 전처리를 진행합니다.")  
    print("-" * 30)  
  
# 실제 브랜드 이름을 인수로 전달하여 함수 호출  
analyze_ppl("세종우유")  
analyze_ppl("한솔제과")
```

🔍 코드 훑아보기

- **brand** : 함수 외부에서 들어올 브랜드 이름을 담아두기 위해 준비된 매개변수입니다.
- **analyze_ppl(세종우유)** : 세종우유라는 실제 문자열 데이터 즉 인수를 함수로 전달하여 맞춤형 출력 결과를 만듭니다.

[3차시] return을 이용한 계산 결과 반환

📌 핵심 개념 설명

단순히 화면에 글자를 보여주고 끝나는 print와 달리 함수가 연산한 최종 결과값을 프로그램의 다른 부분에서 활용해야 할 때가 많습니다. 이때 return 키워드를 사용하면 계산된 수치를 함수 밖으로 꺼내어 변수에 저장할 수 있습니다. PPL 광고의 인지도 점수와 구매 의도 점수를 합산하여 돌려주는 함수를 살펴봅시다.

📄 전체 실습 코드

```
def calculate_total_score(recognition, intent):  
    total = recognition + intent  
    return total  
  
# 계산 결과를 반환받아 변수에 저장  
final_score = calculate_total_score(45, 38)  
  
print("PPL 광고 종합 평가 점수:", final_score)  
print("가중치 반영 점수:", final_score * 1.5)
```

🔍 코드 훑아보기

- **return total** : recognition과 intent를 더한 값을 함수를 호출했던 위치로 돌려주고 함수를 즉시 종료합니다.
- **final_score = ...** : 함수가 돌려준 숫자 83이 final_score 변수에 저장되므로 이후 1.5를 곱하는 등 추가 연산에 바로 활용할 수 있습니다.

[4차시] 위치 인수와 키워드 인수 활용

📌 핵심 개념 설명

함수에 여러 개의 매개변수가 있을 때 값을 전달하는 방식은 두 가지입니다. 코드를 작성한 순서대로 들어가는 위치 인수 방식과 매개변수의 이름을 직접 지정하여 순서와 상관없이 전달하는 키워드 인수 방식입니다. 인구 통계 데이터를 다루는 함수로 두 방식의 차이를 알아봅시다.

📄 전체 실습 코드

```
def population_info(city, district, count):
    print("지역:", city)
    print("행정동:", district)
    print("인구수:", count, "명")
    print("-" * 20)

# 1. 위치 인수: 작성된 순서대로 매핑됨
population_info("세종특별자치시", "한솔동", 18000)

# 2. 키워드 인수: 이름표를 붙여 전달하므로 순서가 바뀌어도 안전함
population_info(count=32000, city="세종특별자치시", district="보람동")
```

🔍 코드 훑아보기

- **위치 인수 호출** : 첫 번째 인수는 city 두 번째는 district 세 번째는 count에 순서대로 엄격하게 들어갑니다.
- **키워드 인수 호출** : 매개변수 이름을 명시하였으므로 count를 가장 먼저 적어도 정확하게 count 매개변수로 값이 들어갑니다.

[5차시] 기본값(Default) 매개변수의 적용

📌 핵심 개념 설명

매번 동일한 값을 전달해야 하는 번거로움을 줄이기 위해 함수 정의 시 매개변수에 기본값을 미리 설정해 둘 수 있습니다. 호출할 때 값을 생략하면 이 기본값이 작동하며 새로운 값을 전달하면 기본값을 무시하고 덮어씁니다.

📄 전체 실습 코드

```
def report_activity(name, club="데이터마이닝"):
    print("이름:", name)
    print("소속 동아리:", club)
    print("-" * 20)

# 두 번째 인수를 생략하여 기본값 적용
report_activity("김학생")
report_activity("이학생")

# 두 번째 인수에 새로운 값을 전달하여 기본값 덮어쓰기
report_activity("박학생", "로봇공학")
```

🔍 코드 훑아보기

- **club=데이터마이닝** : 함수를 만들 때 기본 소속을 데이터마이닝으로 고정해 두었습니다.
- **report_activity(김학생)** : club 자리에 아무것도 넣지 않았으므로 파이썬이 알아서 데이터마이닝을 출력합니다.

[6차시] 지역 변수와 전역 변수의 이해

📌 핵심 개념 설명

함수 안에서 새롭게 만들어진 변수는 지역 변수라 불리며 함수가 종료되면 메모리에서 완전히 사라집니다. 반면 함수 밖에서 만들어진 변수는 전역 변수로 프로그램 전체에서 읽을 수 있습니다. 변수의 생명 주기와 유효 범위를 이해하는 것은 논리 오류를 막는 핵심입니다.

📄 전체 실습 코드

```
base_population = 380000 # 전역 변수

def add_new_residents(new_people):
    # 함수 안에서 생성된 지역 변수 added
    added = base_population + new_people
    print("새로운 인구 유입 후 계산 결과:", added)

add_new_residents(5000)

print("원래 기준 인구수:", base_population)
# print(added) # 주석을 풀면 NameError가 발생합니다. 함수가 끝나면 added는 사라집니다.
```

🔍 코드 훑아보기

- **base_population** : 함수 외부 최상단에 선언되었으므로 함수 내부에서도 접근하여 사용할 수 있는 전역 변수입니다.
- **added** : 함수 내부에서 태어난 지역 변수이므로 밖에서 출력하려 하면 정의되지 않았다는 NameError가 발생합니다.

[7차시] 함수와 조건문(if)의 결합

📌 핵심 개념 설명

입력받은 매개변수의 값을 if 조건문으로 검사하여 참(True) 또는 거짓(False)을 반환하는 판별 함수를 만들 수 있습니다. 데이터의 유효성을 검사하거나 조건에 따른 처리를 모듈화할 때 매우 유용합니다.

📄 전체 실습 코드

```
def is_effective_ppl(score):
    # 점수가 80점 이상이면 광고 효과가 높다고 판별
    if score >= 80:
        return True
    else:
        return False

# 메인 코드에서 함수 활용
current_score = 85

if is_effective_ppl(current_score):
    print("광고 효과가 훌륭합니다. 캠페인을 유지하세요.")
else:
    print("광고 효과가 부족합니다. 매체를 변경하세요.")
```

🔍 코드 톨아보기

- **if score >= 80:** : 외부에서 전달받은 점수를 기준으로 내부에서 조건을 판별합니다.
- **return True** : 연산 결과에 따라 불리언(Boolean) 값을 반환하므로 메인 코드의 if문 조건식에 직관적으로 결합할 수 있습니다.

[8차시] 함수와 반복문(for)의 결합

📌 핵심 개념 설명

여러 개의 데이터가 담긴 파이썬 리스트를 함수의 매개변수로 한 번에 넘겨줄 수 있습니다. 함수 내부의 for 반복문을 통해 리스트 원소를 하나씩 꺼내어 누적 합계를 구하거나 평균을 계산하는 등 대량의 데이터를 손쉽게 다룹니다.

📄 전체 실습 코드

```
def calculate_total_population(pop_list):
    total = 0
    for pop in pop_list:
        total = total + pop
    return total

# 4개 행정동의 인구 데이터 리스트
district_populations = [18000, 32000, 25000, 15000]

# 리스트를 통째로 함수에 전달
total_pop = calculate_total_population(district_populations)
print("행정동 전체 누적 인구수:", total_pop, "명")
```

🔍 코드 훑아보기

- **for pop in pop_list:** 전달받은 리스트에서 데이터를 순차적으로 하나씩 꺼내어 반복문을 실행합니다.
- **total = total + pop:** 초기값이 0인 total 변수에 꺼낸 인구수를 계속해서 누적 합산합니다.

[9차시] 다중 함수의 파이프라인 연결

📌 핵심 개념 설명

복잡한 코드를 하나의 거대한 함수에 몰아넣기보다는 역할이 하나로 분명한 여러 개의 작은 함수로 나누어 설계하는 것이 좋습니다. 한 함수가 연산하여 반환한 결과값을 다음 함수의 입력으로 전달하며 협업하는 구조를 익힙니다.

📄 전체 실습 코드

```
# 1단계: 평균을 구하는 전문 함수
def get_average(scores):
    return sum(scores) / len(scores)

# 2단계: 평균을 받아 등급을 매기는 전문 함수
def rate_effectiveness(avg_score):
    if avg_score >= 80:
        return "우수"
    elif avg_score >= 60:
        return "보통"
    else:
        return "미흡"

ppl_scores = [88, 92, 75, 80]

# 함수들의 파이프라인 협업
average_val = get_average(ppl_scores)
final_rating = rate_effectiveness(average_val)

print("평균 인지도:", average_val)
print("최종 광고 등급:", final_rating)
```

🔍 코드 훑아보기

- **get_average / rate_effectiveness** : 계산 기능과 판별 기능을 분리하여 코드를 수정하고 디버깅하기 훨씬 수월하게 설계했습니다.
- **rate_effectiveness(average_val)** : get_average가 계산해 낸 결과값을 변수에 잠시 담아둔 뒤 이를 그대로 다음 함수의 매개변수로 넘겨줍니다.

[10차시] 종합 미니 프로젝트 - 데이터 분석 리포트 완성

📌 핵심 개념 설명

지금까지 배운 매개변수, 반환값, 조건문, 반복문, 함수 연결의 모든 개념을 조립하여 독립적인 기능을 수행하는 데이터 분석 미니 프로그램을 완성합니다.

📄 전체 실습 코드

```
def calc_avg(data_list):
    return sum(data_list) / len(data_list)

def judge_trend(avg):
    if avg >= 80:
        return "상승세 뚜렷"
    elif avg >= 60:
        return "유지 중"
    else:
        return "하락세 주의"

def generate_report(brand_name, recent_data):
    # 앞서 만든 부품 함수들을 활용
    average = calc_avg(recent_data)
    trend = judge_trend(average)

    print("=" * 30)
    print("브랜드명:", brand_name)
    print("최근 데이터 평균치:", round(average, 1))
    print("현재 분석 트렌드:", trend)
    print("=" * 30)

# --- 메인 실행 제어부 ---
target_brand = "세종푸드"
collected_data = [82, 85, 78, 90, 88]

# 함수 단 하나만 호출하여 전체 분석 및 출력 로직을 수행
generate_report(target_brand, collected_data)
```

🔍 코드 훑아보기

- **부품 함수 분리** : 수학적 계산과 조건 판별을 각각 calc_avg와 judge_trend로 독립시켰습니다.
- **generate_report** : 마치 오케스트라의 지휘자처럼 부품 함수들을 호출해 결과를 취합하고 최종 화면 출력을 관장합니다.
- **메인 실행부의 간결함** : 데이터를 모아서 지휘자 함수 하나만 호출하면 모든 과정이 깔끔하게 처리되는 모듈화의 장점을 극대화했습니다.